

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Nur Rahman Fauzan - 5024231069

2026

1 Langkah-Langkah Percobaan

Praktikum Modul 1 dilaksanakan pada 8 November 2026. Seluruh percobaan memakai satu project PlatformIO bernama `modul1-praktikum` di Visual Studio Code. Folder project diletakkan pada path tanpa spasi. Setiap percobaan hanya mengganti isi `platformio.ini` dan file kode di `src/`. Alat dan bahan mengikuti Modul 1: STM32 Blackpill, ST-Link, ESP32 DevKit, pushbutton, resistor 10 kΩ, serta kabel jumper.

1.1 Percobaan 1: STM32 Blackpill (Blink)

1. Mengisi `platformio.ini` dengan konfigurasi berikut, lalu menunggu PlatformIO selesai me-refresh konfigurasi.

Listing 1: platformio.ini Percobaan 1

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menghubungkan ST-Link ke Blackpill sesuai Tabel 1, lalu menghubungkan ST-Link ke komputer melalui USB.

Tabel 1: Sambungan ST-Link ke STM32 Blackpill

Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data debug
SWCLK	SWCLK	Jalur clock debug
GND	GND	Ground bersama
3.3V	3V3	Tegangan target

3. Menulis program Blink pada `src/main.cpp`, lalu menjalankan Build dan Upload.

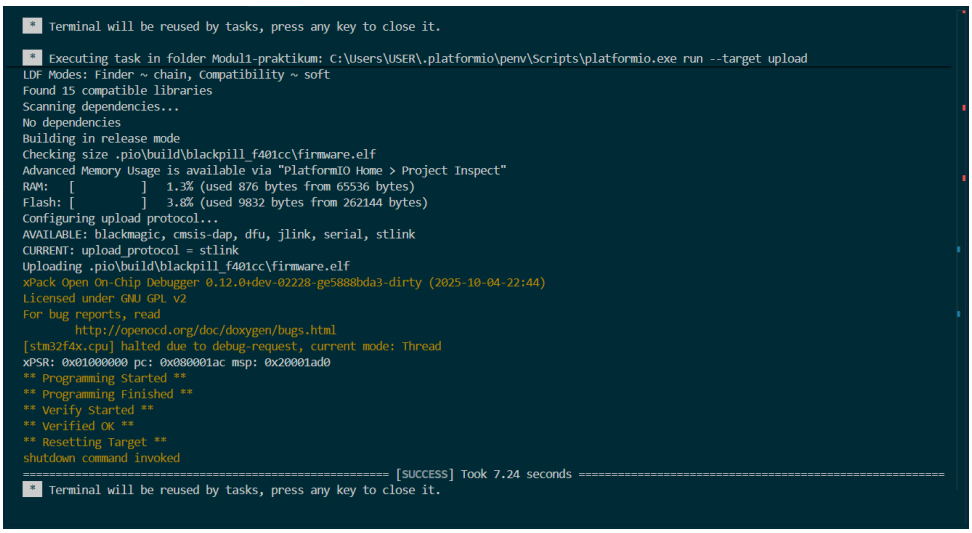
Listing 2: Program Blink STM32 (Arduino)

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW
  delay(500);
  digitalWrite(LED_PIN, HIGH);
  delay(500);
}
```



Gambar 1: Hasil Build dan Upload Percobaan 1 pada terminal PlatformIO

4. Mengamati LED onboard pada pin PC13.

Upload pertama sempat gagal. Penyebab dan penanganannya dibahas pada bagian Analisis Hasil Percobaan.

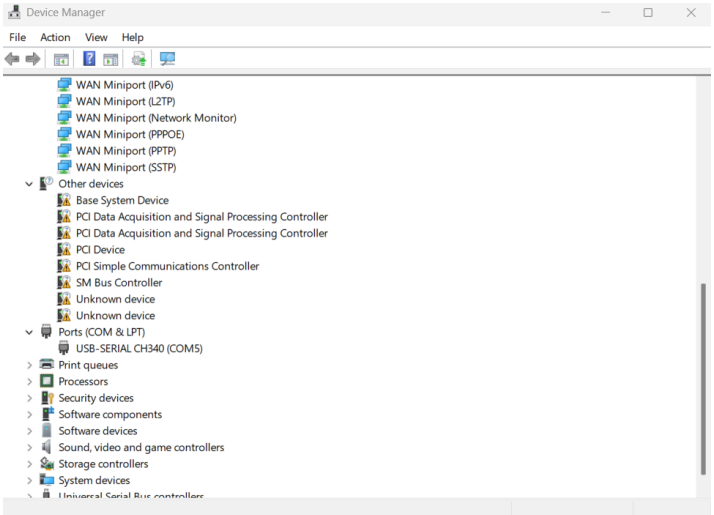
1.2 Percobaan 2: ESP32 dengan Framework ESP-IDF (Blink)

1. Mengganti isi `platformio.ini` dengan konfigurasi ESP-IDF.

Listing 3: platformio.ini Percobaan 2

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Menghubungkan ESP32 ke komputer melalui USB dan memastikan port serialnya terdeteksi (pada kami COM5).



Gambar 2: Device Manager: ESP32 terdeteksi sebagai USB-SERIAL CH340 (COM5)

3. Menghapus `src/main.cpp`, membuat `src/main.c`, menulis program berikut, lalu menjalankan Build dan Upload.

Listing 4: Program Blink ESP32 (ESP-IDF)

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

4. Mengamati LED onboard ESP32 pada GPIO 2.

1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal dan Internal

Bagian A: resistor eksternal.

1. Mengembalikan `platformio.ini` ke framework Arduino, menghapus `src/main.c`, dan membuat kembali `src/main.cpp`. Pada percobaan ini kami menambahkan `monitor_speed = 115200` agar sama dengan `Serial.begin(115200)` pada kode.

Listing 5: platformio.ini Percobaan 3

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
```

2. Merangkai dua pushbutton: tombol pull-up pada GPIO 32 (3V3 melalui resistor 10 kΩ ke node GPIO 32, tombol dari node ke GND) dan tombol pull-down pada GPIO 33 (GND melalui resistor 10 kΩ ke node GPIO 33, tombol dari node ke 3V3).

3. Menulis Kode Program A, menjalankan Build dan Upload, membuka serial monitor, lalu menekan kedua tombol bergantian.

Listing 6: Kode Program A: pull-up dan pull-down eksternal

```c
#include <Arduino.h>

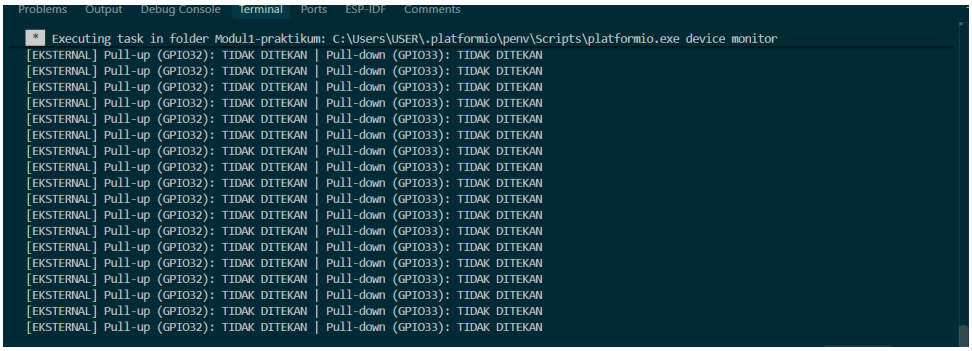
```

```
#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```



Gambar 3: Keluaran serial monitor Kode Program A setelah wiring diperbaiki (tombol tidak ditekan)

Bagian B: resistor internal.

1. Melepas kedua resistor eksternal, tombol tetap terpasang pada GPIO 32 (kaki lain ke GND) dan GPIO 33 (kaki lain ke 3V3).

2. Mengganti isi `src/main.cpp` dengan Kode Program B, menjalankan Build dan Upload, lalu menekan kedua tombol.

Listing 7: Kode Program B: pull-up dan pull-down internal

```
#include <Arduino.h>

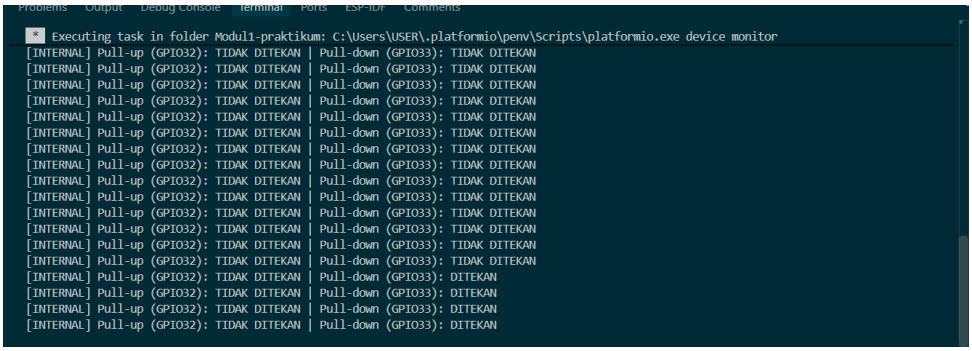
#define BUTTON_PULLUP_PIN 32   // pull-up internal
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
```

```
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```



Gambar 4: Keluaran serial monitor Kode Program B: tombol tidak ditekan, kemudian tombol pull-down (GPIO 33) ditekan

1.4 Percobaan 4 dan 5

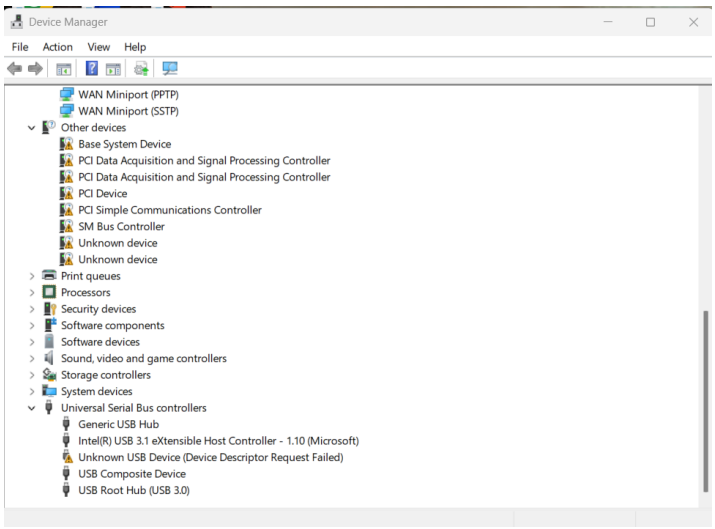
Percobaan 4 (debouncing) dan Percobaan 5 (ADC dengan LDR) tidak sempat dilaksanakan karena waktu praktikum habis setelah Percobaan 3. Oleh karena itu laporan ini tidak memuat langkah, data, maupun analisis hasil untuk kedua percobaan tersebut.

2 Analisis Hasil Percobaan

2.1 Percobaan 1

LED onboard PC13 menyala 500 ms lalu mati 500 ms secara berulang, sehingga periodenya 1000 ms (frekuensi 1 Hz) dengan duty cycle 50%. Hasil ini sesuai dengan program: dua pemanggilan `delay(500)` dalam satu putaran `loop()`. LED onboard Blackpill bersifat aktif LOW, yaitu anodanya terhubung ke 3,3 V dan katodanya ke pin PC13, sehingga LED menyala ketika pin diberi LOW. Karena itu `digitalWrite(LED_PIN, LOW)` menyalakan LED dan `HIGH` mematikannya, sesuai dengan teori pada modul.

Upload pertama gagal dengan pesan OpenOCD `Error: open failed`, dan Device Manager menampilkan *Unknown USB Device (Device Descriptor Request Failed)*. Build sudah berhasil, jadi kesalahan bukan pada kode, melainkan pada komunikasi komputer dengan ST-Link melalui USB. Kami memasang driver ST-Link (paket `stsw-link009`), tetapi perangkat masih bertanda seru di Device Manager. Setelah sambungan kabel SWD diperiksa dan kesalahan wiring diperbaiki, upload berhasil. Pelajaran dari kendala ini adalah urutan pemeriksaan: kode (build berhasil atau tidak), driver, lalu sambungan fisik.



Gambar 5: Device Manager menampilkan *Unknown USB Device (Device Descriptor Request Failed)* saat upload pertama gagal

2.2 Percobaan 2

LED onboard pada GPIO 2 berkedip dengan periode 1000 ms, sama seperti Percobaan 1, sehingga program ESP-IDF terbukti berjalan di ESP32. Karena sama-sama Blink, perbedaan Arduino dan ESP-IDF terlihat pada struktur programnya (Tabel 2).

Tabel 2: Perbandingan program Blink dengan Arduino dan ESP-IDF

Aspek	**Arduino**	**ESP-IDF**
Titik masuk	`setup()` sekali, `loop()` berulang	`app_main()`, perulangan ditulis sendiri
Konfigurasi pin	`pinMode()`	`gpio_reset_pin()` dan `gpio_set_direction()`
Menulis level pin	`digitalWrite()`	`gpio_set_level()`
Penundaan	`delay()` (ms)	`vTaskDelay(pdMS_TO_TICKS())`
Berkas utama	`main.cpp`	`main.c`

ESP-IDF menuntut lebih banyak baris dan pemahaman API tingkat rendah, tetapi memberi akses langsung ke FreeRTOS dan fitur ESP32. Arduino lebih ringkas karena detail tersebut disembunyikan oleh framework. Pada ESP32, framework Arduino sendiri berjalan di atas FreeRTOS, sehingga perbedaan keduanya terutama pada tingkat abstraksi dan kontrol, bukan pada ada atau tidaknya RTOS.

Ada dua hal teknis yang memengaruhi keberhasilan build ESP-IDF di PlatformIO. Pertama, saat persiapan sebelum praktikum, project yang berada pada path dengan spasi gagal dikonfigurasi (*Detected a whitespace character in project paths*), sehingga project dipindahkan ke path tanpa spasi. Kedua, program ESP-IDF harus disimpan sebagai `main.c`. Pada file C++ nama fungsi mengalami *name mangling*, sehingga `app_main` tidak dikenali kecuali dideklarasikan `extern "C"`.

2.3 Percobaan 3

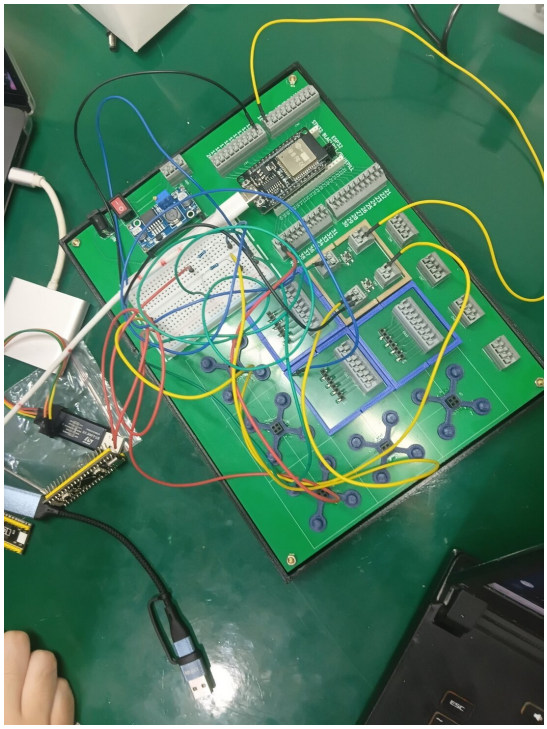
Tabel 3 merangkum perilaku yang diharapkan menurut teori, dan keluaran serial monitor pada kode memetakannya menjadi tulisan DITEKAN atau TIDAK DITEKAN.

Tabel 3: Level pin yang diharapkan pada Percobaan 3

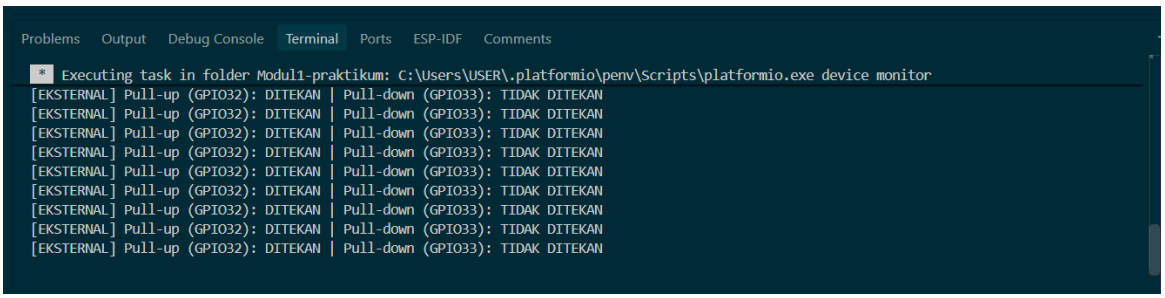
Kondisi tombol	Pull-up (GPIO 32)	Pull-down (GPIO 33)
Tidak ditekan	HIGH (TIDAK DITEKAN)	LOW (TIDAK DITEKAN)
Ditekan	LOW (DITEKAN)	HIGH (DITEKAN)

Pada pull-up, resistor menarik pin ke 3,3 V saat tombol terbuka, dan tombol menghubungkan pin ke GND saat ditekan, sehingga logikanya aktif LOW. Pada pull-down, resistor menarik pin ke GND saat tombol terbuka, dan tombol menghubungkan pin ke 3,3 V saat ditekan, sehingga logikanya aktif HIGH. Kode memakai `stateUp == LOW` dan `stateDown == HIGH` untuk menyesuaikan perbedaan logika ini.

Pada percobaan awal serial monitor menampilkan GPIO 32 (pull-up) selalu DITEKAN walaupun tombol tidak disentuh, artinya pin terbaca LOW terus. Hal ini konsisten dengan pin yang tidak tertarik ke 3,3 V dan/atau tersambung ke GND akibat kesalahan wiring, sehingga rangkaian tidak berfungsi sebagai pull-up. Asisten praktikum memperbaiki wiring tersebut tanpa penjelasan rinci, dan kami tidak mencatat bagian mana yang diubah, sehingga penyebab pastinya tidak dapat kami tentukan. Setelah wiring diperbaiki, GPIO 32 (pull-up) dan GPIO 33 (pull-down) terbaca TIDAK DITEKAN saat kedua tombol tidak ditekan, sesuai Tabel 3 (tangkapan layar pada Langkah Percobaan 3, Bagian A). Pembacaan saat tombol ditekan pada Bagian A tidak terdokumentasi dengan bukti yang dapat dipastikan, sehingga kondisi DITEKAN yang benar tidak dapat kami tunjukkan dari tangkapan layar.



Gambar 6: Rangkaian pada trainer board saat GPIO 32 terbaca DITEKAN terus (sebelum wiring diperbaiki)



Gambar 7: Serial monitor saat GPIO 32 (pull-up) terbaca DITEKAN terus walaupun tombol tidak ditekan

Pull-up dan pull-down diperlukan agar pin input tidak *floating*. Pin yang mengambang memiliki impedansi sangat tinggi sehingga levelnya tidak terdefinisi dan dipengaruhi noise di sekitarnya. Kejadian di atas menunjukkan bahwa ketika jalur pull-up tidak benar, pembacaan bisa salah secara konsisten, bukan hanya acak.

Resistor eksternal 10 kΩ dan resistor internal memiliki fungsi yang sama tetapi berbeda dalam praktik. Dengan 3,3 V, arus ke GND saat tombol ditekan pada pull-up 10 kΩ adalah $3{,}3/10.000 = 0{,}33$ mA. Resistor pull internal ESP32 bernilai sekitar 45 kΩ (nilai tipikal pada datasheet), sehingga arusnya sekitar 0,07 mA. Resistor internal menghemat komponen dan kabel, tetapi tarikannya lebih lemah sehingga lebih rentan noise pada kabel yang panjang. Resistor eksternal lebih kuat dan nilainya dapat dipilih. Pada Kode Program B (pin dan tombol yang sama), serial monitor menampilkan GPIO 32 (pull-up internal) TIDAK DITEKAN dan GPIO 33 (pull-down internal) TIDAK DITEKAN saat tombol tidak disentuh, lalu GPIO 33 berubah menjadi DITEKAN saat tombol pull-down ditekan. Pola ini sama dengan Kode Program A dan sesuai teori bahwa `INPUT_PULLUP` dan `INPUT_PULLDOWN` mengaktifkan resistor yang berfungsi sama dengan resistor eksternal, sehingga rangkaian tetap bekerja walaupun resistor 10 kΩ dilepas. Tangkapan layar yang kami miliki tidak memuat pembacaan GPIO 32 (pull-up) saat ditekan, sehingga bagian itu tidak dapat kami dokumentasikan.

2.4 Percobaan 4 dan 5

Kedua percobaan tidak dilaksanakan sehingga tidak ada data yang dianalisis. Pengaruh nilai waktu debounce (10, 50, dan 200 ms) dan pembacaan ADC dengan LDR belum kami uji langsung. Teori debouncing dan ADC baru diterapkan pada Tugas Modul, itu pun dalam simulasi.

3 Hasil Tugas Modul

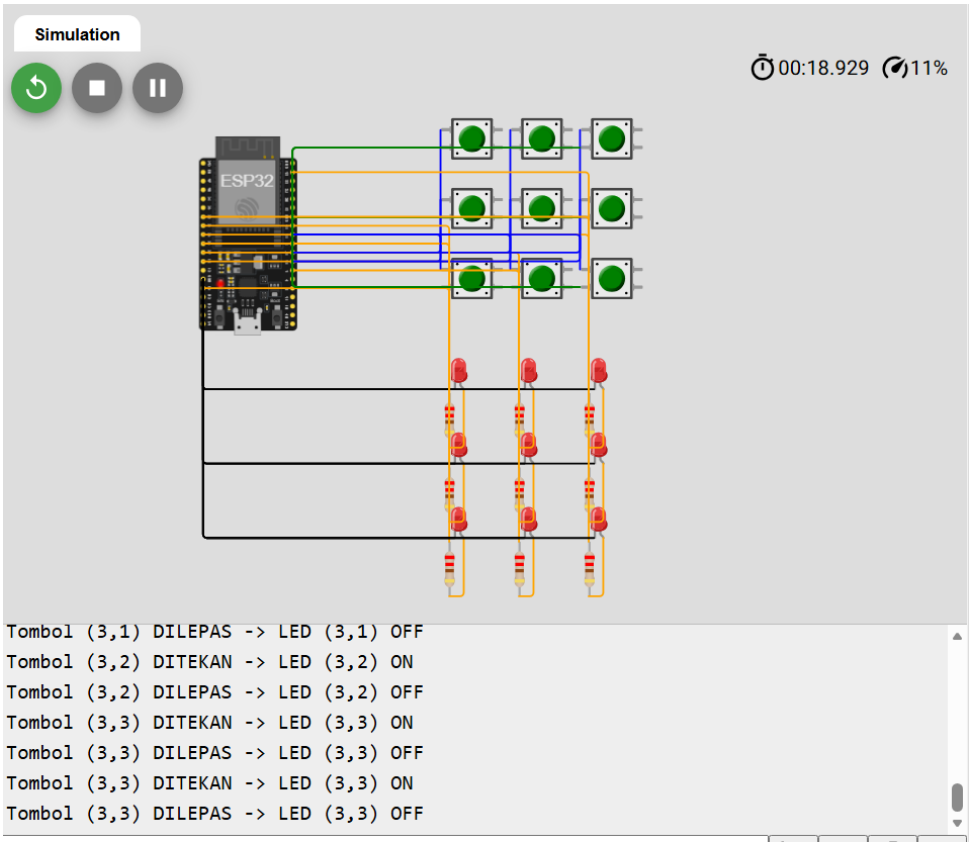
3.1 Rancangan

Tugas modul meminta simulasi matriks pushbutton 3x3 yang mengendalikan matriks LED 3x3, dengan tombol pada posisi (baris, kolom) menyalakan LED pada posisi yang sama dan wajib memakai debouncing. Simulasi dibuat di Wokwi dengan ESP32 DevKit. Matriks tombol dibaca dengan teknik *scanning*: baris menjadi output dan kolom menjadi input dengan pull-up internal. Matriks LED dibuat dari sembilan LED yang masing-masing memakai satu pin GPIO dan resistor 220 Ω, dengan susunan posisi yang sama seperti tombol. Pemetaan pin ditunjukkan Tabel 4.

Tabel 4: Pemetaan pin matriks 3x3

Fungsi	Baris/Kolom 1	Baris/Kolom 2	Baris/Kolom 3
Baris tombol (output)	GPIO 16	GPIO 17	GPIO 18
Kolom tombol (input pull-up)	GPIO 19	GPIO 21	GPIO 22
LED baris 1	GPIO 13	GPIO 14	GPIO 25
LED baris 2	GPIO 26	GPIO 27	GPIO 32
LED baris 3	GPIO 33	GPIO 4	GPIO 23

Pada setiap putaran scanning, satu baris ditarik ke LOW sementara baris lain dibiarkan *high-Z* (mode input). Jika tombol pada baris aktif ditekan, kolomnya terbaca LOW. Debouncing dilakukan per tombol: pembacaan mentah dibandingkan dengan pembacaan sebelumnya, dan setiap perubahan mengulang penghitungan waktu. Status tombol baru diterima dan LED diubah hanya setelah pembacaan stabil selama `DEBOUNCE_MS` = 30 ms. LED mengikuti status tombol yang sudah stabil, yaitu menyala selama tombol ditekan.



Gambar 8: Rangkaian simulasi Wokwi saat tidak ada tombol ditekan: ESP32, matriks pushbutton 3x3, dan matriks LED 3x3 (seluruh LED mati)

3.2 Kode Program

Listing 8: Kode program matriks pushbutton 3x3 dengan debouncing

```
#include <Arduino.h>

// ===== Matrix Pushbutton 3x3 -> Matrix LED 3x3 (ESP32, framework Arduino) =====
// Baris tombol  : OUTPUT (diaktifkan LOW satu per satu saat scanning)
// Kolom tombol  : INPUT_PULLUP (pull-up internal, tanpa resistor eksternal)
// LED           : satu pin per LED (resistor 220 ohm), posisi sama dengan tombol
```

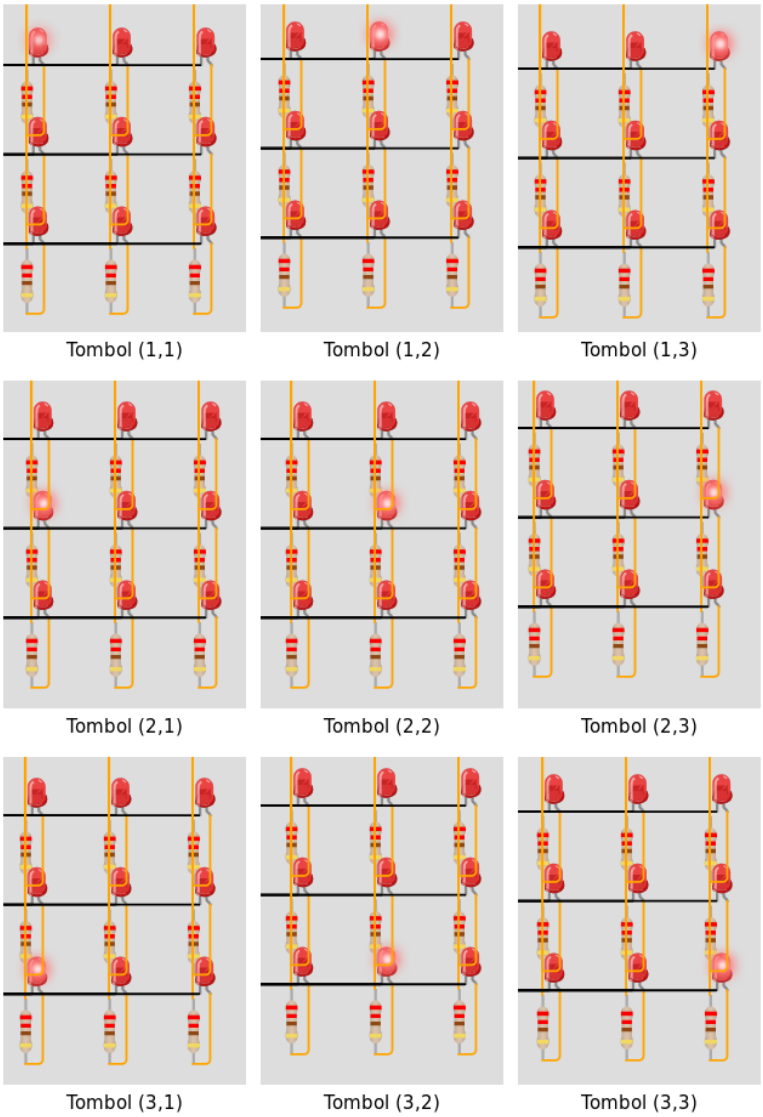
```
7
8  const uint8_t ROW_PINS[3] = {16, 17, 18};
9  const uint8_t COL_PINS[3] = {19, 21, 22};
10 const uint8_t LED_PINS[3][3] = {
11     {13, 14, 25},
12     {26, 27, 32},
13     {33, 4, 23}};
14
15 const uint32_t DEBOUNCE_MS = 30; // waktu minimum sinyal harus stabil
16
17 bool rawState[3][3];       // pembacaan mentah terakhir (true = ditekan)
18 bool stableState[3][3];    // status stabil, sudah lolos debouncing
19 uint32_t lastChange[3][3]; // waktu terakhir pembacaan mentah berubah
20
21 void setup()
22 {
23   Serial.begin(115200);
24
25   for (int r = 0; r < 3; r++)
26   {
27     pinMode(ROW_PINS[r], INPUT); // baris dibiarkan high-Z sampai giliran di-scan
28   }
29   for (int c = 0; c < 3; c++)
30   {
31     pinMode(COL_PINS[c], INPUT_PULLUP); // tidak ditekan = HIGH, ditekan = LOW
32   }
33   for (int r = 0; r < 3; r++)
34   {
35     for (int c = 0; c < 3; c++)
36     {
37       pinMode(LED_PINS[r][c], OUTPUT);
38       digitalWrite(LED_PINS[r][c], LOW);
39       rawState[r][c] = false;
40       stableState[r][c] = false;
41       lastChange[r][c] = 0;
42     }
43   }
44   Serial.println("Matrix pushbutton 3x3 siap");
45 }
46
47 void loop()
48 {
49   uint32_t now = millis();
50
51   for (int r = 0; r < 3; r++)
52   {
53     // Aktifkan baris r: tarik ke LOW, baris lain tetap high-Z
54     pinMode(ROW_PINS[r], OUTPUT);
55     digitalWrite(ROW_PINS[r], LOW);
56     delayMicroseconds(10); // beri waktu sinyal settle sebelum dibaca
57
58     for (int c = 0; c < 3; c++)
59     {
60       bool raw = (digitalRead(COL_PINS[c]) == LOW); // LOW = kontak tertutup
```

```
61
62        // Pembacaan mentah berubah -> mulai hitung ulang waktu stabil
63        if (raw != rawState[r][c])
64        {
65          rawState[r][c] = raw;
66          lastChange[r][c] = now;
67        }
68
69        // Sudah stabil minimal DEBOUNCE_MS dan berbeda dari status sah -> terima
70        if ((now - lastChange[r][c]) >= DEBOUNCE_MS && stableState[r][c] != rawState[r
      ][c])
71        {
72          stableState[r][c] = rawState[r][c];
73          digitalWrite(LED_PINS[r][c], stableState[r][c] ? HIGH : LOW);
74          Serial.printf("Tombol (%d,%d) %s -> LED (%d,%d) %s\n",
75                        r + 1, c + 1, stableState[r][c] ? "DITEKAN" : "DILEPAS",
76                        r + 1, c + 1, stableState[r][c] ? "ON" : "OFF");
77        }
78      }
79
80      pinMode(ROW_PINS[r], INPUT); // lepas baris r sebelum scan baris berikutnya
81    }
82  }
```

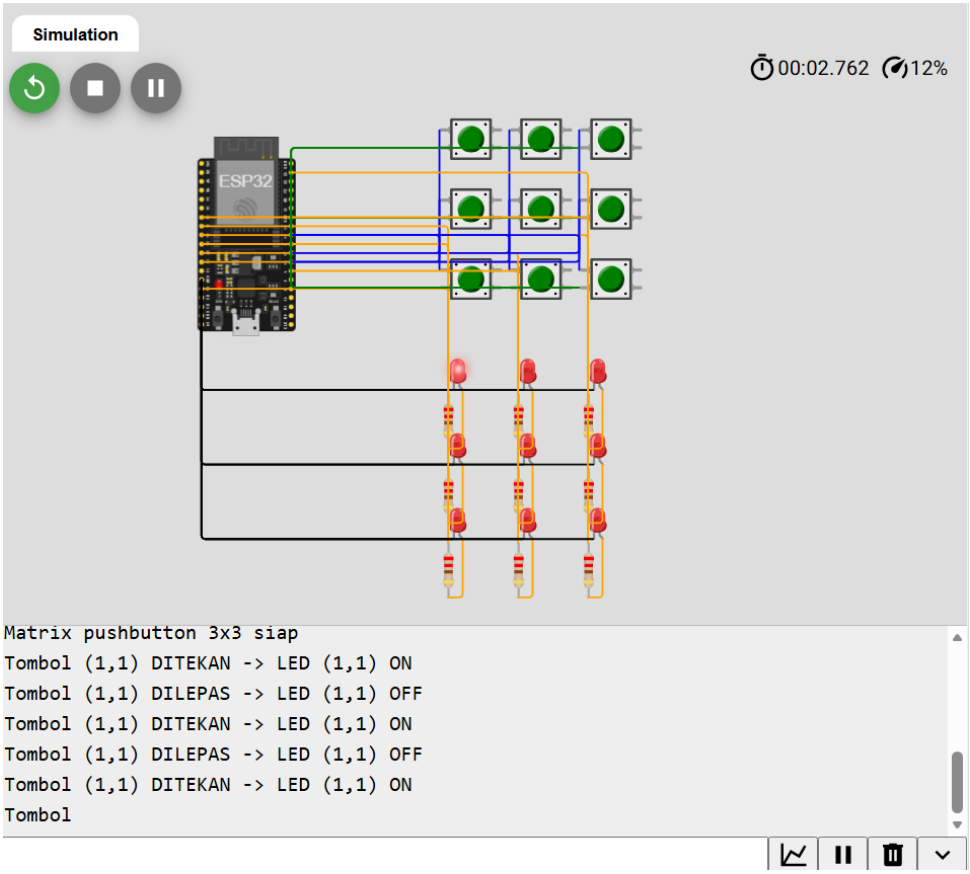
3.3 Hasil Simulasi

Simulasi dijalankan di Wokwi dan kesembilan tombol diuji satu per satu dengan cara ditekan lalu dilepas. Pada setiap pengujian, LED pada posisi (baris, kolom) yang sama dengan tombol menyala selama tombol ditekan dan padam ketika tombol dilepas, sedangkan delapan LED lainnya tetap mati. Seluruh sembilan pasangan tombol dan LED sesuai dengan pemetaan pada Tabel 4. Serial monitor mencatat setiap perubahan status, yaitu satu baris `DITEKAN` saat tombol ditekan dan satu baris `DILEPAS` saat dilepas, tanpa baris ganda untuk satu kali penekanan.

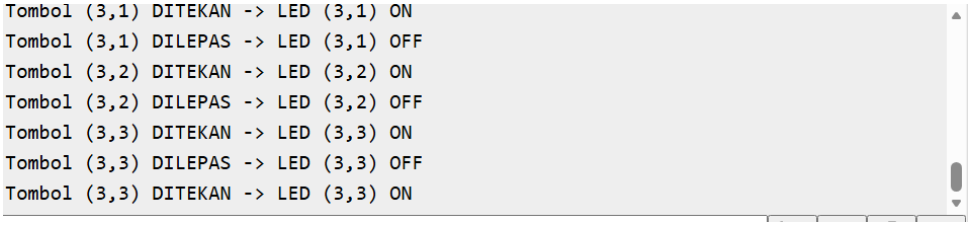
Batasan pengujian: tombol hanya ditekan satu per satu. Penekanan beberapa tombol sekaligus tidak diuji. Selain itu, pushbutton pada simulator tidak memodelkan pantulan kontak mekanik, sehingga manfaat debouncing tidak dapat diamati langsung dalam simulasi ini dan hanya didasarkan pada logika program serta penjelasan pada jawaban analisis.



Gambar 9: Matriks LED pada simulasi untuk kesembilan tombol yang ditekan satu per satu: hanya LED dengan posisi yang sama yang menyala



Gambar 10: Simulasi dengan tombol baris 1 kolom 1 ditekan: LED baris 1 kolom 1 menyala dan serial monitor mencatat kejadian



Gambar 11: Keluaran serial monitor saat tombol ditekan dan dilepas

3.4 Jawaban Pertanyaan Analisis

1. **Mengapa debouncing tetap diperlukan?** Bouncing adalah sifat fisik setiap kontak mekanik, bukan akibat banyaknya tombol yang ditekan. Pada saat kontak tersambung atau terlepas, pegas logam memantul beberapa kali dalam waktu singkat (orde milidetik) sehingga sinyal bolak-balik antara HIGH dan LOW sebelum stabil, seperti dijelaskan pada dasar teori debouncing. Matriks 3x3 hanya mengatur cara pembacaan, sedangkan setiap tombol tetap berupa satu kontak mekanik yang memantul. Loop scanning membaca tombol sangat cepat, jauh lebih cepat daripada durasi pantulan, sehingga satu kali penekanan akan terbaca sebagai beberapa perubahan. Tanpa debouncing, LED berkedip atau berubah status mengikuti pantulan, dan pada program yang menghitung kejadian satu penekanan dihitung berkali-kali. Dengan menunggu sinyal stabil selama 30 ms, pantulan diabaikan dan LED hanya berubah satu kali per penekanan. Gangguan singkat (*glitch*) yang lebih pendek dari 30 ms juga tidak diterima.

2. **Konfigurasi pull-up pada pembacaan kolom.** Kolom dikonfigurasi `INPUT_PULLUP`, yaitu resistor internal menarik pin kolom ke 3,3 V. Saat tombol tidak ditekan, pin kolom terbaca HIGH. Saat tombol ditekan dan barisnya sedang di-scan (LOW), kolom tersambung ke baris sehingga terbaca LOW, jadi logikanya aktif LOW. Kondisi HIGH ini tidak boleh dibiarkan *floating* karena pin input CMOS memiliki impedansi sangat tinggi, sehingga levelnya tidak terdefinisi dan mudah berubah akibat noise, induksi dari kabel, atau muatan liar. Pembacaan yang tidak menentu membuat tombol terdeteksi tertekan padahal tidak, sehingga LED berkedip sendiri. Hal serupa kami amati pada Percobaan 3: ketika jalur pull-up tidak terpasang dengan benar, GPIO 32 terbaca LOW terus dan tombol dianggap selalu ditekan.

3. **Digital input dan analog input (ADC).** Pada digital input, mikrokontroler hanya membandingkan tegangan pin dengan ambang logika dan menghasilkan dua kondisi, HIGH atau LOW. Pada ESP32 ambang tersebut sekitar 0,25 dan 0,75 dari tegangan catu (nilai kira-kira dari datasheet), dan tegangan di antaranya tidak memberi informasi tambahan. Pada analog input, ADC mengukur besar tegangan dan mengubahnya menjadi bilangan. ADC 12-bit pada ESP32 membagi rentang sekitar 0 sampai 3,3 V menjadi $2^{12} = 4096$ tingkat (0 sampai 4095), dengan resolusi sekitar $3{,}3/4095 \approx 0{,}81$ mV per tingkat. Bila sebuah LED diganti LDR dalam rangkaian pembagi tegangan dengan resistor 10 kΩ, tegangan keluaran berubah kontinu mengikuti hambatan LDR, yang bergantung pada intensitas cahaya. ADC memetakan tegangan itu ke salah satu dari 4096 nilai, sehingga dapat menyatakan banyak tingkat terang, bukan hanya terang atau gelap. Perlu dicatat bahwa ADC ESP32 tidak sepenuhnya linear di dekat ujung rentang.

4 Kesimpulan

Praktikum Modul 1 berhasil dijalankan sampai Percobaan 3. Program Blink berjalan pada STM32 Blackpill dengan framework Arduino melalui ST-Link (LED PC13 aktif LOW) dan pada ESP32 dengan framework ESP-IDF (LED GPIO 2). Kedua percobaan sesuai teori: periode kedipan 1000 ms mengikuti dua `delay` 500 ms. Kendala upload pada ST-Link ternyata berasal dari sambungan fisik, bukan dari kode, dan pada ESP-IDF path project harus tanpa spasi serta file program harus berekstensi `.c`.

Pada Percobaan 3, pembacaan saat tombol tidak ditekan sesuai teori (pull-up HIGH dan pull-down LOW), dan kesalahan wiring terbukti dapat membuat pembacaan salah secara konsisten (GPIO 32 selalu terbaca tertekan). Secara teori pull-up berlogika aktif LOW dan pull-down aktif HIGH. Pada kondisi yang terdokumentasi, resistor internal ESP32 (sekitar 45 kΩ) menghasilkan pembacaan yang sama dengan resistor eksternal 10 kΩ, dengan perbedaan pada kekuatan tarikan dan jumlah komponen.

Percobaan 4 dan 5 tidak dilaksanakan karena waktu praktikum habis, sehingga kesimpulan mengenai pengaruh nilai debounce dan pembacaan ADC tidak dapat diambil dari percobaan langsung. Pada Tugas Modul, simulasi di Wokwi menunjukkan bahwa scanning baris-kolom dengan pull-up internal dapat memetakan kesembilan tombol ke kesembilan LED dengan benar, dengan logika debouncing per tombol berjendela stabil 30 ms. Pengaruh debouncing terhadap pantulan kontak belum teramati karena simulator tidak memodelkan pantulan, dan hasil simulasi ini belum diuji pada perangkat keras.

5 Lampiran

5.1 Dokumentasi saat praktikum

Foto dokumentasi kegiatan praktikum tidak tersedia. Bukti pelaksanaan percobaan berupa tangkapan layar pada bagian Langkah-Langkah Percobaan dan Hasil Tugas Modul.